Nesne Yönelimli Programlama

Kısa Tanım Soruları

Sekiz konunun sonundaki "What is it?" tanımları tek yerde

10 Kasım 2026

Nasıl kullanılır?

- Bu sayfa, her konu anlatımının sonundaki **Kısa tanım soruları** bölümünü bir araya toplar; aynı tanımlar konu sayfalarının sonunda da duruyor (iki yer aynı kaynaktan üretilir, birbirinden farklılaşmaz).
- Sınavda "What is …?", "Define …" ya da test sorusu olarak gelirler (ör. *regression fault, moving target, modern maintenance, cohesion, coupling, actor, use case, aggregation*). Tanımlar sınavda yazılacağı gibi **İngilizce**dir.
- Kendini sınamak için: terimi oku, tanımı kapatıp kendi cümlenle İngilizce yaz, sonra karşılaştır. Ayrıntısı için ilgili konunun sayfasına bak.

1 Yazılım Mühendisliğinin Kapsamı (Chapter 1)

Software crisis Software is delivered late, over budget and with residual faults.

Regression fault A fault in an apparently unrelated part of the software, caused by a change made elsewhere. Information hiding reduces regression faults.

Regression testing Re-running all previous test cases after a change to check that nothing else is broken.

Moving target problem A change in the requirements while the software is being developed (Chapter 2).

Verification / validation Verification: testing at the end of each phase. Validation: testing at the end of the project (whether the product satisfies the specification).

Modern maintenance Maintenance occurs whenever software is modified, before or after installation.

Object A software component that incorporates both data and the actions performed on that data.

Mistake / fault / failure / error A mistake is made by a human; its consequence is a fault in the code; a failure is the observed incorrect behavior caused by the fault; the error is the amount by which a result is incorrect (IEEE 610.12).

Information hiding Hiding the implementation details of a module so that other modules cannot depend on them; future changes are localized.

Encapsulation Gathering everything that relates to a real-world item (data and actions) into one unit.

Responsibility-driven design Sending a message to an object without knowing how it performs the task (design by contract).

2 Yaşam Döngüsü Modelleri (Chapter 2)

Moving target problem A change in the requirements while the software product is being developed. It induces dependencies and can cause regression faults. There is no solution; its effects can only be minimized.

Regression fault A fault in an apparently unrelated part of the software caused by a change.

Miller's law At any one time we can concentrate on only about seven chunks of information.

Stepwise refinement Concentrate on the currently most important aspects and postpone the less critical ones; every aspect is eventually handled in order of importance.

Iteration Each successive version is closer to the target than its predecessor.

Incrementation Adding new functionality step by step; the product is built as a set of increments (mini projects).

Baseline A complete set of artifacts at a point in time (e.g. at the end of an episode).

Robustness The property of being able to handle extensions and changes without falling apart.

3 Unified Process (Chapter 3)

Workflow vs phase A workflow is the technical context of a step; a phase is the business context of a step. Every step falls into one workflow and one phase.

Business case A document describing the scope of the product and its financial details (revenue projections, cost estimates or cost–benefit analysis); its initial version is the overall aim of the inception phase.

Beta release The first operational-quality version produced at the end of the construction phase.

Traceability Every item in an artifact must be traceable to an item in the previous artifact.

Specification document A contract between client and developer; it must not contain contradictions, omissions or incompleteness.

Three major risks Technical risks, the risk of not getting the requirements right, and the risk of not getting the architecture right.

4 Modüller, Cohesion ve Coupling (Chapter 7)

Module A lexically contiguous sequence of program statements, bounded by boundary elements, with an aggregate identifier.

Cohesion The degree of interaction within a module. (Seven levels: coincidental, logical, temporal, procedural, communicational, functional, informational.)

Coupling The degree of interaction between two modules. (Five levels: content, common, control, stamp, data.)

Good design High cohesion and low coupling (maximal interaction within a module, minimal interaction between modules).

Data encapsulation A data structure together with the operations performed on that data structure.

Abstract data type A data type together with the operations performed on instantiations of that data type (a module with informational cohesion; a class).

Information hiding Designing modules so that items likely to change are hidden; future change is localized.

Class An abstract data type that supports inheritance.

Dynamic binding The method to be invoked is determined at run time according to the class of the object.

Polymorphism The same method can be applied to objects of different classes, each behaving in its own way.

Fragile base class problem A change to a base class ripples through all of its subclasses.

5 ISO/IEC 25010 Kalite Modeli

ISO/IEC 25010 The quality model of the ISO/IEC 25000 (SQuaRE) series; it defines which quality characteristics are taken into account when evaluating a software product (2023: nine characteristics, 40 sub-characteristics).

Quality (of a system) The degree to which the system satisfies the stated and implied needs of its various stakeholders.

Functional suitability The degree to which a product provides functions that meet stated and implied needs (completeness, correctness, appropriateness).

Performance efficiency Performance relative to the amount of resources used (time behaviour, resource utilization, capacity).

Compatibility The degree to which a product can exchange information with other products and share the same environment (co-existence, interoperability).

Interaction capability The degree to which a product can be interacted with by specified users (formerly usability: learnability, operability, user error protection, inclusivity…).

Reliability The degree to which a system performs specified functions under specified conditions for a specified period of time (faultlessness, availability, fault tolerance, recoverability).

Security The degree to which a product protects information and data according to the level of authorization (confidentiality, integrity, non-repudiation, accountability, authenticity, resistance).

Non-repudiation Actions or events can be proven to have taken place, so they cannot be denied later.

Maintainability The degree of effectiveness and efficiency with which a product can be modified (modularity, reusability, analysability, modifiability, testability).

Flexibility The degree to which a product can be adapted to changes in its requirements, contexts of use or environment (formerly portability: adaptability, scalability, installability, replaceability).

Safety The degree to which a product avoids states that endanger human life, health, property or the environment (operational constraint, risk identification, fail safe, hazard warning, safe integration).

6 Gereksinimler ve Use Case (Chapter 11)

Use case Models an interaction between the software product itself and the users of that software product (actors).

Actor A member of the world outside the software product; a role (user, initiator or someone who plays a critical part). It need not be a human.

Business model A description of the business processes of an organization.

Glossary A list of technical words used in the domain, and their meanings.

Structured / unstructured interview Pre-planned, frequently close-ended questions / questions posed in response to the answers received, frequently open-ended.

«include» Behavior factored out of a use case for reuse; the base use case always performs it. Arrow from the base use case to the included use case.

«extend» Exceptional or seldom invoked cases factored out of the main flow. Arrow from the extending use case to the extended (base) use case.

Functional / nonfunctional requirement An action the product must perform / a property of the product itself (platform, response time, reliability).

7 Nesne Yönelimli Analiz (Chapter 13)

Entity / boundary / control class Models long-lived information / models the interaction between the product and its environment (input, output, reports) / models complex computations and algorithms.

Scenario An instance of a use case.

Noun extraction Describe the product in a single paragraph, identify the nouns, discard those outside the problem boundary and the abstract nouns; the rest are candidate classes.

CRC card Class–Responsibility–Collaboration card: the name of the class, its responsibilities and the classes it invokes.

God class A class that is exposed to too much information and has too much control (an antipattern).

Use-case realization The process of extending and refining use cases; the realization of a scenario is depicted by a sequence or collaboration diagram.

Collaboration diagram Shows the objects, the links between them and the messages numbered in the order in which they are sent.

Sequence diagram Shows the objects and the messages between them in time order (top to bottom); calls are solid arrows, returns dashed arrows.

8 UML: Sınıf İlişkileri ve Diyagramlar

UML A standard graphical modeling language (OMG) for specifying, visualizing, constructing and documenting software; not a process and independent of the implementation language.

Association A relationship between instances of two classes, where one class must know about the other to do its work (solid line; arrow if one-directional).

Aggregation An association where one class belongs to a collection (collection–member); hollow diamond on the collection side; no sole ownership.

Composition A strong form of aggregation with lifetime control: the parts cannot exist without the whole; filled diamond on the whole side.

Inheritance An “is-a” relationship; a hollow triangle pointing to the superclass.

Realization A class implements an interface; dashed line with a hollow triangle pointing to the interface.

Dependency A "uses-a" relationship; a change in one class may affect the other; dashed open arrow.

Multiplicity The number of instances of one class that can be associated with one instance of the other (1, 0..1, *, 1..*, n..m).